



OA

▼ Netradyne

▼ DigitSum till one digit left

```
int solve(string &str,int &k){
    int sum =0;
    for(int i=0;i<str.length();i+=k){
        sum+=str[i]-'0';
    }
    if(sum<10) return sum;
    string s = to_string(sum);
    return solve(s,k);
}
```

▼ Josephus

```
//iterative soln when indexing are 1 based (1 to n)
int Josephus(int n,int k){
    int ans=0;
    for(int i=1;i<=n;i++){
        ans = (ans+k)%i;
    }
    return ans+1;
}

//iterative soln when indexing are 0 based (0 to n-1)
int Josephus(int n,int k){
    int ans=0;
    for(int i=1;i<=n;i++){
        ans = (ans+k)%i;
    }
}
```

```

    }
    return ans;
}

```

▼ hiring frontend and backend with min cost (n+m)

```

int solve(vector<int> &f,vector<int> &b,int n,int m){
    vector<pair<int,int>> temp;
    for(int i=0;i<n+m;i++){
        temp.push_back({f[i],b[i]});
    }
    int ans=0;
    sort(temp.begin(),temp.end(),[&](const pair<int,int> &a,
        const pair<int,int> &b){
        return a.first-a.second < b.first-b.second;
    });
    for(int i=0;i<n+m;i++){
        if(i<n){
            ans+=temp[i].first;
        }
        else{
            ans+=temp[i].second;
        }
    }
    return ans;
}

```

▼ Max Displacement

```

int solve(string &str,int &n){ // n changes u have to do
    int len = str.size();
    int sc = 0;
    for(auto &it:str){
        if(it=='S') sc++;
    }
    int mn = min(sc,len-sc);
}

```

```

int mx = len - mn;
if(n<=mn){
    return mx+2*n-mn; //mx + n - (mn - n), for the case
}
if((n-mn)%2==0){
    return len;
}
return len-1;
}

```

▼ Min of extreme elements of a subsequence of size m

```

int solve(vector<int> &v,int m){
    int n=v.size();
    vector<int> preMin(n);
    preMin[0]=v[0];
    vector<int> sufMin(n);
    sufMin[n-1]=v[n-1];
    for(int i=1;i<n;i++){
        preMin[i]=min(preMin[i-1],v[i]);
    }
    for(int i=n-2;i>=0;i--){
        sufMin[i]=min(sufMin[i+1],v[i]);
    }
    int ans =INT_MAX;
    int i=0;
    int j=m-1;
    while(i<=n-m){
        ans = min(ans,preMin[i]*sufMin[j]);
        i++;
        j++;
    }
    return ans;
}

```

▼ make all elements of an array equal in a given range (L to R)

```
int solve(vector<int> &v,int l,int r){
    int mx=0;
    for(int i=l;i<=r;i++){
        mx=(mx | v[i]);
    }
    int temp=1;
    int ans=0;
    while(temp<=mx){
        int cnt =0;
        if((mx & temp)==temp){ // if in the mx result the
            for(int i=l;i<=r;i++){
                if((temp & v[i])==0) cnt++; // if bit is 0
            }
        }
        ans+=cnt;
        temp= temp*2;
    }
    return ans;
}
```